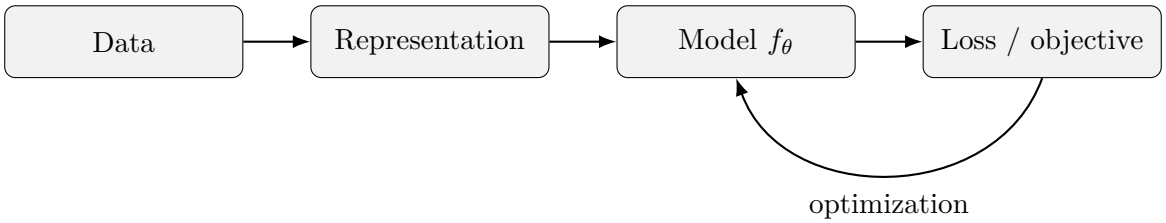
Machine Learning & Artificial Intelligence

Lecture Notes

From data and statistics to neural networks, simulation-based inference, and large language models



Audience: Advanced high-school students who are comfortable with algebra, graphs, functions, and basic scientific reasoning, but who may have little formal background in statistics, calculus, or computer science.

Central idea: Machine learning is a mathematical/statistical framework for learning useful mappings, structures, or distributions from data. Artificial intelligence is a broader field concerned with constructing systems that perform tasks associated with intelligence. Modern frontier chatbots are overwhelmingly built from machine-learning methods, but AI is not logically identical to ML: symbolic, rule-based, planning, search, and other approaches can also be part of AI.

Prepared by Nathan Steinle

October 13, 2026

Contents

1 What are machine learning and artificial intelligence? 5

1.1 Artificial intelligence is broader than machine learning 5

1.2 A taxonomy that keeps the ideas straight 6

1.3 What modern chatbots actually are 6

2 The central mathematical problem in machine learning 7

2.1 The unknown mapping 7

2.2 Why the data problem is fundamental 7

2.3 Parameters versus hyperparameters 8

3 Regression and classification 8

3.1 Regression 8

3.2 Classification 9

3.3 Why probability matters 9

4 Supervised, unsupervised, self-supervised, and reinforcement learning 9

4.1 Supervised learning 9

4.2 Unsupervised learning 10

4.3 Self-supervised learning 10

4.4 Reinforcement learning 10

5 Statistics needed to understand ML 10

5.1 Probability 10

5.2 Mean, variance, and standard deviation 11

5.3 Likelihood versus probability 11

6 The machine-learning objective: loss, risk, and optimization 12

6.1 Loss for one example 12

6.2 Empirical risk 12

6.3 Gradient descent 13

6.4 A complete one-parameter example 13

7 Neural networks: the core mathematical construction **14**

7.1 The artificial neuron . . . 14

7.2 Layers and composition . . . 15

7.3 Diagram of a small neural network . . . 15

7.4 Why neural networks can approximate complicated mappings . . . 15

8 A fully worked neural-network calculation **16**

8.1 Network definition . . . 16

8.2 Compute the loss . . . 17

8.3 Backpropagation: output layer . . . 17

8.4 Backpropagation: hidden layer . . . 17

8.5 One gradient-descent update . . . 18

9 How do we know the network is learning? **18**

9.1 Train, validation, and test sets . . . 19

9.2 Overfitting . . . 19

9.3 Learning curves . . . 19

9.4 Classification diagnostics . . . 20

9.5 Calibration . . . 20

10 Representation and the geometry of parameter space **21**

10.1 Features as coordinates . . . 21

10.2 Standardization . . . 21

10.3 Principal component analysis (PCA) . . . 21

10.4 A two-dimensional intuition . . . 22

10.5 Learned latent spaces with neural networks . . . 22

10.5.1 Why this is analogous to PCA . . . 24

10.5.2 What “latent” means . . . 24

10.5.3 Denoising autoencoders: forcing the representation to be useful . . . 25

10.5.4 Variational autoencoders: a probabilistic latent space . . . 25

10.5.5 Why latent spaces are useful in science . . . 26

11 Generalization: why complicated models can still work **28**

11.1 Distribution shift 28

12 A compact survey of common ML algorithms 29

13 Simulation-based inference and simulation-based calibration 30

13.1 Inverse problem intuition 30

13.2 Normalizing flows 30

13.3 Simulation-based calibration (SBC) 31

14 Examples from astrophysics 31

14.1 1. Simulation-based calibrated Bayesian inference for binary stellar evolution 31

14.2 2. Normalizing flows and simulation-based inference for multimessenger inverse problems 32

14.3 3. ML classification in the black-hole mass–spin plane 32

15 From machine learning to today's large language models 34

15.1 Tokens 34

15.2 Embeddings 34

15.3 Attention 34

15.4 Why the Transformer matters 35

15.5 Scale 35

15.6 Human feedback and instruction following 35

16 What other kinds of AI exist? 36

17 AI competition, current systems, and the US–China comparison 36

18 Should we fear AI? A scientifically useful way to discuss the question 37

19 A unifying picture: ML as function learning 37

20 Compact summary for students 39

21 Key equations 40

22 Questions for discussion or assessment 40

References **41**

Note on the astrophysics examples **42**

1 What are machine learning and artificial intelligence?

1.1 Artificial intelligence is broader than machine learning

There is no single universally accepted definition of artificial intelligence (AI). The difficulty is not merely semantic: different definitions focus on human-like thought, rational action, perception, problem solving, autonomy, or the observable behavior of an engineered system. The *Stanford Encyclopedia of Philosophy* reviews several competing formulations and emphasizes that AI has long combined technical, engineering, and philosophical questions [1].

A practical engineering definition used by the U.S. National Institute of Standards and Technology (NIST) describes an AI system as an engineered or machine-based system that, for given objectives, generates outputs such as predictions, recommendations, or decisions that influence real or virtual environments, with varying levels of autonomy [3].

Machine learning (ML), by contrast, is a more specific computational idea: use data or experience to estimate a function, probability distribution, representation, or decision rule whose exact form is not supplied completely by hand. A classical formulation from Mitchell [5] defines a machine-learning system in terms of a task T, a performance measure P, and experience E: a program learns from E with respect to T and P if its performance at T, measured by P, improves with experience E.

> **Expert insight.** A useful mental model is: *AI is a broad goal/field; ML is a family of mathematical and computational mechanisms for getting machines to improve or infer from data.* The relationship is therefore not "AI = ML." Rather, much of today's most capable practical AI is implemented using ML.

The often-useful "atoms versus molecules" analogy should be used carefully. Machine learning is not literally the atom of AI, because some AI systems can be constructed without machine learning. Historically, many AI systems relied on explicit rules, symbolic logic, search, planning, and manually encoded knowledge [1, 2]. The analogy becomes reasonable when discussing *modern data-driven AI*: ML supplies many of the core mathematical engines inside contemporary systems.

1.2 A taxonomy that keeps the ideas straight

Concept	Question it asks	Typical mathematical object
AI	How can we build systems that perform tasks associated with intelligence?	An engineered system with objectives, inputs, outputs, and behavior
Machine learning	How can a system infer useful rules/representations from data or experience?	A learned function/distribution f_θ, parameters θ, objective $J(\theta)$
Statistics	How should we quantify uncertainty and infer patterns from data?	Probability distributions, estimators, likelihoods, confidence/credible intervals
Data science	How do we turn data into reliable information and decisions?	A workflow combining statistics, computing, domain knowledge, and visualization
Deep learning	How can multi-layer neural networks learn useful representations?	Compositions of parameterized nonlinear functions

Important distinction. It is therefore better to say "ML is a central mathematical approach used in modern AI" than "ML and AI are the same thing" or "all AI is ML."

1.3 What modern chatbots actually are

Modern large language models (LLMs) are machine-learning systems based principally on deep neural networks. The Transformer architecture, introduced in 2017, replaced recurrent sequence processing with attention-based computation and became a foundation for modern language modeling [7]. Training commonly occurs in stages. A simplified picture is:

1. **Pretraining:** the model sees very large amounts of text and learns to predict tokens from context. This is often called "self-supervised learning" because the target labels are automatically generated from the data itself (for next-token prediction, the next token is the target).
2. **Post-training / instruction tuning:** supervised examples and preference data can be used to make model behavior better aligned with instructions. InstructGPT, for example, used supervised fine-tuning followed by reinforcement learning from human feedback (RLHF) [8].
3. **Inference-time systems:** the model may be surrounded by retrieval, tool use, safety classifiers, routing, structured-output systems, memory systems, and other software.

These surrounding components need not all be neural networks.

Important distinction. "Unsupervised learning" is often used loosely when describing LLM pretraining. More precise terminology is *self-supervised learning*: the training targets are constructed automatically from the input data. This distinction matters because the task still has an explicit prediction target.

2 The central mathematical problem in machine learning

2.1 The unknown mapping

Suppose an input is represented by a vector

$$\boldsymbol{x} = \begin{bmatrix} x_1 & x_2 & \cdots & x_d \end{bmatrix}^{\mathsf{T}} \in \mathbb{R}^d,$$

and the desired output is y.

In the simplest supervised-learning picture, there is some unknown relationship

$$y \approx f^*(\boldsymbol{x}),$$

where f^* is the real-world mapping we wish to approximate.

We do not know f^* exactly. Instead, we observe examples

$$\mathcal{D} = \left\{ (\boldsymbol{x}^{(1)}, y^{(1)}), \ldots, (\boldsymbol{x}^{(N)}, y^{(N)}) \right\}.$$

Machine learning chooses a model family $\{f_\theta\}$ and learns parameters θ so that

$$f_\theta(\boldsymbol{x}) \approx f^*(\boldsymbol{x}).$$

Here θ can mean a small number of parameters (for example, slope and intercept in a line) or billions of parameters in a large neural network.

2.2 Why the data problem is fundamental

A model cannot reliably learn a mapping in regions of input space for which the training data contain no useful information, unless some additional prior structure or inductive bias bridges the gap.

Imagine one scalar input x and suppose the training data occupy only

$$0 \leq x \leq 1.$$

A model may fit the data extremely well on that interval but behave almost arbitrarily for $x = 10$. The issue is not necessarily that the algorithm is "bad." The information needed to determine the mapping at $x = 10$ may simply not be present.

For d independent input dimensions, a naive grid requiring m samples along each dimension needs roughly

$$N \sim m^d$$

points. This exponential growth is one way of seeing the *curse of dimensionality*: exhaustive coverage of a high-dimensional space rapidly becomes impossible.

> **Expert insight.** The deepest limitation of machine learning is often not the optimization algorithm. It is the relationship between the information contained in the data, the structure imposed by the model, and the region of parameter/input space over which we want the model to generalize.

2.3 Parameters versus hyperparameters

A **parameter** is a quantity the model learns from the training data. For a linear model

$$\widehat{y} = wx + b,$$

w and b are parameters.

A **hyperparameter** is a setting chosen by the experimenter or outer optimization procedure, such as learning rate, number of hidden layers, tree depth, or regularization strength.

Confusing these concepts makes the training process difficult to reason about: the model learns θ, while we normally select hyperparameters using validation performance.

3 Regression and classification

3.1 Regression

Regression predicts a numerical quantity. Examples include predicting temperature, a galaxy property, a continuous probability, or a physical parameter.

A simple linear regression model is

$$\widehat{y} = wx + b.$$

For data points (x_i, y_i), a common objective is the mean squared error (MSE):

$$J(w, b) = \frac{1}{N} \sum_{i=1}^{N} (wx_i + b - y_i)^2.$$

The algorithm seeks values of w and b that make this quantity small.

3.2 Classification

Classification predicts membership in a discrete category.

For binary classification,

$$y \in \{0, 1\}.$$

A model often produces a number $p \in [0, 1]$ interpreted as an estimated probability:

$$p_\theta(y = 1 \mid \boldsymbol{x}).$$

A common choice is the logistic (sigmoid) function

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

If $z = 0$, then

$$\sigma(0) = \frac{1}{2}.$$

As $z \to +\infty$, $\sigma(z) \to 1$; as $z \to -\infty$, $\sigma(z) \to 0$.

A model can therefore define

$$p = \sigma(\boldsymbol{w} \cdot \boldsymbol{x} + b).$$

A classification rule can be obtained with a threshold, for example

$$\hat{y} = \begin{cases} 1, & p \geq 0.5, \\ 0, & p < 0.5. \end{cases}$$

3.3 Why probability matters

Suppose two cases produce $p = 0.51$ and $p = 0.99$. Both would be classified as class 1 by a 0.5 threshold, but they do not express the same degree of model confidence. Therefore, classification quality should not be judged only by the final class label. Metrics such as log loss, calibration, precision, recall, area under a ROC curve, and confusion matrices can matter depending on the application.

4 Supervised, unsupervised, self-supervised, and reinforcement learning

4.1 Supervised learning

The training set contains input-output examples:

$$\{(\boldsymbol{x}_i, y_i)\}_{i=1}^{N}.$$

The objective is to learn a mapping from $\boldsymbol{x}$ to y.

Examples:

- regression: predict a continuous value;
- classification: predict one of several classes.

4.2 Unsupervised learning

There are no externally supplied labels y_i. The algorithm attempts to find structure in the inputs themselves.

Examples include clustering, dimensionality reduction, density estimation, and representation learning.

4.3 Self-supervised learning

The data generate their own targets. In next-token prediction, for example, a sequence

"The cat sat on the"

can be turned into a training example whose target is the next token appearing in the original sequence.

This is supervised optimization mathematically—the model predicts a target and minimizes a loss—but the labels are derived automatically rather than manually assigned by a human.

4.4 Reinforcement learning

An agent interacts with an environment. It observes a state s, takes an action a, receives a reward r, and seeks behavior that produces high long-term reward.

A simple objective is to maximize expected discounted return

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots,$$

where $0 \leq \gamma \leq 1$ is a discount factor.

RLHF is one historical method for using human preferences to shape a language model. It should not be confused with the entire field of reinforcement learning, and modern post-training uses a broader collection of preference-optimization methods.

5 Statistics needed to understand ML

5.1 Probability

Probability quantifies uncertainty. For events A and B,

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}, \qquad \mathbb{P}(B) > 0.$$

Bayes' theorem follows by writing the joint probability in two ways:

$$\mathbb{P}(A \cap B) = \mathbb{P}(A \mid B)\mathbb{P}(B)$$

and also

$$\mathbb{P}(A \cap B) = \mathbb{P}(B \mid A)\mathbb{P}(A).$$

Therefore,

$$\boxed{\mathbb{P}(A \mid B) = \frac{\mathbb{P}(B \mid A)\mathbb{P}(A)}{\mathbb{P}(B)}}.$$

The same idea underlies Bayesian inference:

$$p(\theta \mid D) = \frac{p(D \mid \theta)p(\theta)}{p(D)}.$$

Here $p(\theta)$ is the prior, $p(D \mid \theta)$ is the likelihood, and $p(\theta \mid D)$ is the posterior.

5.2 Mean, variance, and standard deviation

For numbers $x_1, \ldots, x_N$, the sample mean is

$$\bar{x} = \frac{1}{N}\sum_{i=1}^{N} x_i.$$

The sample variance is

$$s^2 = \frac{1}{N-1}\sum_{i=1}^{N}(x_i - \bar{x})^2,$$

and the sample standard deviation is

$$s = \sqrt{s^2}.$$

Variance measures how widely the values spread around the mean. In ML, scaling features by their mean and standard deviation can be important when an algorithm depends on distances or gradient magnitudes.

5.3 Likelihood versus probability

A useful conceptual distinction is:

- probability asks, "If the model parameters were known, how plausible is this data?"
- likelihood treats the observed data as fixed and asks, "Which parameter values make these data most plausible?"

For independent data points,

$$p(D \mid \theta) = \prod_{i=1}^{N} p(y_i \mid \boldsymbol{x}_i, \theta).$$

It is usually easier to maximize the log-likelihood:

$$\log p(D \mid \theta) = \sum_{i=1}^{N} \log p(y_i \mid \boldsymbol{x}_i, \theta).$$

Products become sums, which are numerically and algebraically convenient.

6 The machine-learning objective: loss, risk, and optimization

6.1 Loss for one example

A loss function $\ell(y, \widehat{y})$ measures how wrong a prediction is.

For squared error,

$$\ell(y, \widehat{y}) = (\widehat{y} - y)^2.$$

For binary cross-entropy,

$$\ell(y, p) = -\left[y \ln p + (1 - y) \ln(1 - p)\right].$$

Check the two cases explicitly.

If $y = 1$:

$$\ell = -\ln p.$$

As $p \to 1$, $\ell \to 0$; as $p \to 0$, $\ell \to \infty$.

If $y = 0$:

$$\ell = -\ln(1 - p).$$

The same logic applies with the roles reversed.

6.2 Empirical risk

For a dataset of N examples,

$$J(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell\left(y_i, f_\theta(\boldsymbol{x}_i)\right).$$

This is the empirical average loss. The training algorithm attempts to minimize it.

A more conceptual quantity is the population risk,

$$\mathcal{R}(\theta) = \mathbb{E}_{(\boldsymbol{X},Y)\sim p_{\text{data}}}\left[\ell\left(Y, f_\theta(\boldsymbol{X})\right)\right],$$

which is the expected loss on new draws from the real data-generating distribution.

The central challenge is that we can compute the empirical risk from the finite training set, but the population risk is what we actually care about.

> **Expert insight.** Training asks, "Can my model fit these examples?" Generalization asks, "Will the same learned rule work on new examples drawn from the relevant population?" The second question is usually the scientifically important one.

6.3 Gradient descent

Suppose a one-parameter model has objective $J(\theta)$. If the derivative $dJ/d\theta$ is positive, increasing θ increases the objective locally; decreasing θ should reduce it. Therefore the basic update is

$$\boxed{\theta_{\text{new}} = \theta_{\text{old}} - \eta\frac{dJ}{d\theta}},$$

where $\eta > 0$ is the learning rate.

For many parameters,

$$\boxed{\boldsymbol{\theta}_{\text{new}} = \boldsymbol{\theta}_{\text{old}} - \eta\nabla_\theta J}.$$

The gradient points in the direction of steepest increase. Therefore $-\nabla J$ points in the direction of steepest local decrease.

6.4 A complete one-parameter example

Consider

$$y = 2x$$

and a model with no intercept,

$$\widehat{y} = wx.$$

Use a single data point $x = 3$, $y = 6$.

The squared loss is

$$J(w) = (3w - 6)^2.$$

Expand:

$$J(w) = 9w^2 - 36w + 36.$$

Differentiate:

$$\frac{dJ}{dw} = 18w - 36.$$

Suppose $w = 1$ initially. Then

$$\frac{dJ}{dw} = 18(1) - 36 = -18.$$

With learning rate $\eta = 0.10$,

$$w_{\text{new}} = 1 - 0.10(-18) = 2.8.$$

The parameter moved toward the correct value $w = 2$.

7 Neural networks: the core mathematical construction

7.1 The artificial neuron

A neuron first computes a weighted sum,

$$z = \sum_{j=1}^{d} w_j x_j + b = \boldsymbol{w} \cdot \boldsymbol{x} + b,$$

and then applies an activation function ϕ:

$$a = \phi(z).$$

Common activation functions include:

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}},$$

and the rectified linear unit (ReLU),

$$\text{ReLU}(z) = \max(0, z).$$

The point of the nonlinearity is crucial. If we stack only linear transformations,

$$f(\boldsymbol{x}) = W_2(W_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2,$$

then this can be rewritten as another single affine map:

$$f(\boldsymbol{x}) = W\boldsymbol{x} + \boldsymbol{b}.$$

Therefore, multiple linear layers without nonlinear activation do not produce a genuinely more expressive model than one linear layer.

7.2 Layers and composition

A feed-forward neural network is a composition of functions:

$$\boldsymbol{h}^{(1)} = \phi_1(W^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}),$$

$$\boldsymbol{h}^{(2)} = \phi_2(W^{(2)}\boldsymbol{h}^{(1)} + \boldsymbol{b}^{(2)}),$$

$$\cdots$$

$$\widehat{y} = \phi_L(W^{(L)}\boldsymbol{h}^{(L-1)} + \boldsymbol{b}^{(L)}).$$

A network with one or more hidden layers can learn intermediate representations of the input. Deep learning refers broadly to models built from many such layers.

7.3 Diagram of a small neural network

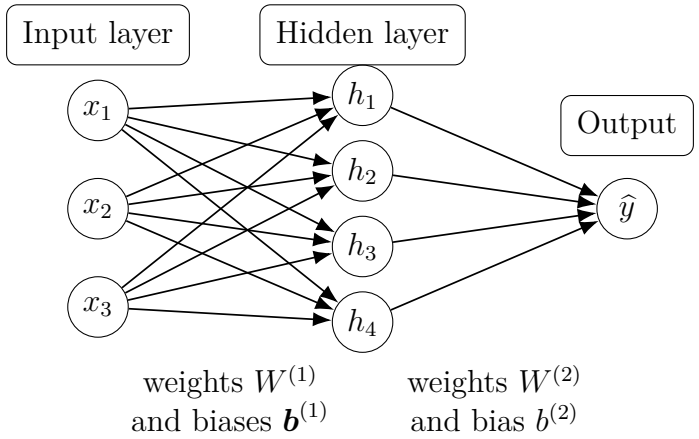


Figure 1: A neural network is a composition of parameterized functions. Each connection carries a learned weight; each neuron has a bias.

7.4 Why neural networks can approximate complicated mappings

A single linear neuron can only produce a hyperplane decision boundary. Nonlinear activations allow the network to bend and combine features. With enough hidden units and suitable conditions, neural networks can approximate broad classes of continuous functions; this is the idea behind universal approximation theorems. However, "can approximate" does not mean "will learn the correct function from finite data." Optimization, data coverage, noise, architecture, and generalization remain essential.

8 A fully worked neural-network calculation

We now do one complete forward-and-backward pass with explicit numbers. The purpose is not to train a useful model by hand; it is to expose the algebra hiding inside a neural network.

8.1 Network definition

Inputs:

$$\boldsymbol{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Target:

$$y = 1.$$

There are two hidden neurons using ReLU and one output neuron using a sigmoid.

Hidden neuron 1:

$$z_1 = 0.5x_1 - 0.5x_2 + 0.1.$$

Substitute:

$$z_1 = 0.5(1) - 0.5(2) + 0.1 = 0.5 - 1 + 0.1 = -0.4.$$

Therefore,

$$h_1 = \mathrm{ReLU}(-0.4) = 0.$$

Hidden neuron 2:

$$z_2 = 1.0x_1 + 0.5x_2 - 0.2.$$

Substitute:

$$z_2 = 1(1) + 0.5(2) - 0.2 = 1 + 1 - 0.2 = 1.8.$$

Therefore,

$$h_2 = \mathrm{ReLU}(1.8) = 1.8.$$

Output pre-activation:

$$z = 0.4h_1 - 0.3h_2 + 0.2.$$

Substitute:

$$z = 0.4(0) - 0.3(1.8) + 0.2 = -0.54 + 0.2 = -0.34.$$

Output probability:

$$p = \sigma(-0.34) = \frac{1}{1 + e^{0.34}} \approx 0.4158.$$

8.2 Compute the loss

Because $y = 1$, binary cross-entropy is

$$L = -\ln p.$$

Hence,

$$L = -\ln(0.4158) \approx 0.877.$$

The model is therefore assigning probability only about 0.416 to the correct class, so the loss is substantial.

8.3 Backpropagation: output layer

For sigmoid output with binary cross-entropy, the derivative simplifies to

$$\frac{\partial L}{\partial z} = p - y.$$

Here,

$$\delta \equiv \frac{\partial L}{\partial z} = 0.4158 - 1 = -0.5842.$$

For the output weight $v_1 = 0.4$,

$$\frac{\partial L}{\partial v_1} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial v_1} = \delta h_1 = (-0.5842)(0) = 0.$$

For $v_2 = -0.3$,

$$\frac{\partial L}{\partial v_2} = \delta h_2 = (-0.5842)(1.8) \approx -1.0516.$$

For the output bias $c = 0.2$,

$$\frac{\partial L}{\partial c} = \delta = -0.5842.$$

8.4 Backpropagation: hidden layer

For hidden neuron 2,

$$\frac{\partial L}{\partial h_2} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial h_2} = (-0.5842)(-0.3) \approx 0.1753.$$

Since $z_2 = 1.8 > 0$,

$$\frac{d}{dz}\,\mathrm{ReLU}(z_2) = 1.$$

Therefore,

$$\frac{\partial L}{\partial z_2} \approx 0.1753.$$

Now use

$$z_2 = w_{21}x_1 + w_{22}x_2 + b_2.$$

Hence,

$$\frac{\partial L}{\partial w_{21}} = 0.1753(1) = 0.1753,$$
$$\frac{\partial L}{\partial w_{22}} = 0.1753(2) = 0.3506,$$
$$\frac{\partial L}{\partial b_2} = 0.1753.$$

Hidden neuron 1 has $z_1 = -0.4 < 0$, so ReLU's derivative is 0 there. Consequently its local gradient is zero for this particular example and first-order update.

8.5 One gradient-descent update

Take $\eta = 0.10$.

For v_2:

$$v_2^{\text{new}} = -0.3 - 0.10(-1.0516) = -0.3 + 0.10516 \approx -0.19484.$$

For c:

$$c^{\text{new}} = 0.2 - 0.10(-0.5842) = 0.25842.$$

For w_{21}:

$$w_{21}^{\text{new}} = 1.0 - 0.10(0.1753) = 0.98247.$$

For w_{22}:

$$w_{22}^{\text{new}} = 0.5 - 0.10(0.3506) = 0.46494.$$

For b_2:

$$b_2^{\text{new}} = -0.2 - 0.10(0.1753) = -0.21753.$$

The important point is not these particular numbers. It is the mechanism:

$$\boxed{\text{forward pass} \rightarrow \text{loss} \rightarrow \text{derivatives} \rightarrow \text{parameter update.}}$$

Backpropagation is essentially an efficient application of the chain rule to this computation graph [6].

9 How do we know the network is learning?

A small training loss is not enough evidence of a scientifically useful model.

9.1 Train, validation, and test sets

A standard workflow partitions data into:

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}} \cup \mathcal{D}_{\text{test}}.$$

- **Training set:** used to fit model parameters.
- **Validation set:** used to choose hyperparameters and compare model variants.
- **Test set:** held back until the end for an approximately unbiased final performance estimate under the test sampling assumptions.

The critical rule is: information from the test set must not leak backward into model selection.

9.2 Overfitting

Suppose a model is flexible enough to memorize the training examples. Then training error can become tiny while the error on new data remains large.

Conceptually:

$$\text{underfitting} \quad \longleftrightarrow \quad \text{good fit} \quad \longleftrightarrow \quad \text{overfitting}$$

corresponds to increasing model flexibility, but the exact relationship depends on the data and training procedure.

Typical defenses include more representative data, regularization, early stopping, data augmentation where appropriate, simpler models, and careful validation.

9.3 Learning curves

A useful diagnostic is to plot training and validation loss versus training step or epoch.

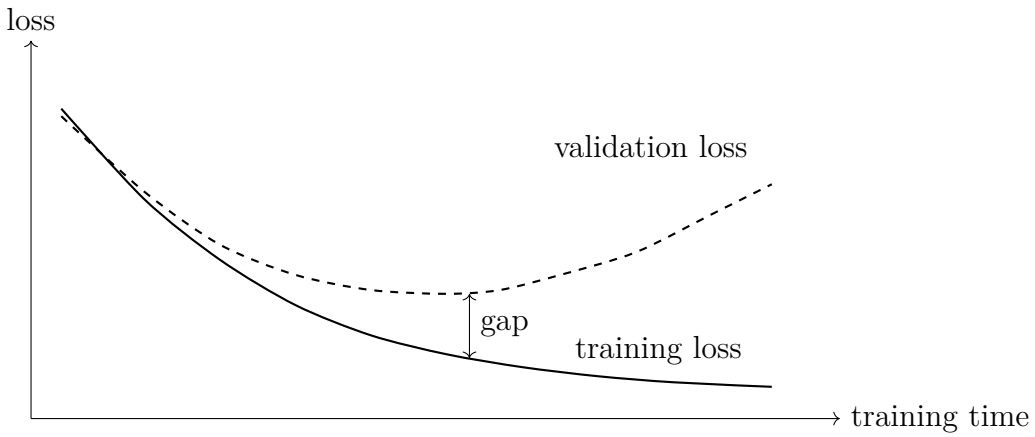


Figure 2: Schematic learning curves. When validation loss begins rising while training loss keeps falling, overfitting may be occurring. The curves are illustrative, not data from a particular experiment.

9.4 Classification diagnostics

For binary classification, define

$$\text{TP} = \text{true positives}, \quad \text{TN} = \text{true negatives},$$
$$\text{FP} = \text{false positives}, \quad \text{FN} = \text{false negatives}.$$

Then

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN},$$
$$\text{precision} = \frac{TP}{TP + FP},$$
$$\text{recall} = \frac{TP}{TP + FN}.$$

Accuracy can be misleading for strongly imbalanced classes. For example, if 99% of examples are class 0, a model that predicts class 0 for every example has 99% accuracy but fails completely on class 1.

9.5 Calibration

A classifier is *calibrated* when predictions claiming probability p are correct about p fraction of the time, under the relevant sampling assumptions.

For example, among many cases to which a model assigns about 0.8 probability of class 1, roughly 80% should actually be class 1 if the probabilities are well calibrated.

This distinction is profound:

$$\text{discrimination} \neq \text{calibration}.$$

A model can rank cases correctly yet produce systematically overconfident probabilities.

10 Representation and the geometry of parameter space

10.1 Features as coordinates

Suppose each object has d measured properties. Then one observation is a point in a d-dimensional feature space:

$$\boldsymbol{x} = (x_1, x_2, \ldots, x_d).$$

Machine learning does not see the physical object in its full philosophical richness. It sees the chosen representation.

If two physically different objects are mapped to the same feature vector, the model cannot distinguish them using those features alone.

> **Expert insight.** Feature representation is often as important as model choice. A sophisticated algorithm cannot recover information that the representation has thrown away.

10.2 Standardization

For a feature x, a common transformation is

$$z = \frac{x - \bar{x}}{s}.$$

After standardization, the feature has sample mean approximately 0 and sample standard deviation approximately 1.

This matters especially for distance-based algorithms and optimization procedures in which differently scaled coordinates can dominate the geometry or gradient.

10.3 Principal component analysis (PCA)

PCA seeks a lower-dimensional linear representation that captures as much variance as possible in the observed data. It is one of the standard dimensionality-reduction techniques in multivariate statistics [10].

Suppose centered data are represented by vectors $\boldsymbol{x}_i \in \mathbb{R}^d$. The sample covariance matrix is

$$C = \frac{1}{N-1} \sum_{i=1}^{N} \boldsymbol{x}_i \boldsymbol{x}_i^{\mathsf{T}}.$$

For a unit vector $\boldsymbol{u}$, the projected coordinate is

$$z_i = \boldsymbol{u}^{\mathsf{T}} \boldsymbol{x}_i.$$

Its sample variance is

$$\mathrm{Var}(z) = \boldsymbol{u}^{\mathsf{T}} C \boldsymbol{u}.$$

To find the first principal component, maximize this variance subject to

$$\boldsymbol{u}^{\mathsf{T}} \boldsymbol{u} = 1.$$

Using a Lagrange multiplier λ,

$$\mathcal{L}(\boldsymbol{u}, \lambda) = \boldsymbol{u}^{\mathsf{T}} C \boldsymbol{u} - \lambda(\boldsymbol{u}^{\mathsf{T}} \boldsymbol{u} - 1).$$

Set the derivative with respect to $\boldsymbol{u}$ to zero:

$$2C\boldsymbol{u} - 2\lambda \boldsymbol{u} = 0.$$

Therefore,

$$C\boldsymbol{u} = \lambda \boldsymbol{u}.$$

Thus the principal-component directions are eigenvectors of the covariance matrix, and the corresponding eigenvalues describe the variance captured along those directions.

10.4 A two-dimensional intuition

If data form a long thin cloud of points, the first principal component points along the long axis. The second component is perpendicular to it. Replacing (x_1, x_2) with (z_1, z_2) can preserve most of the variation while representing the data in a more convenient coordinate system.

PCA is linear. Neural networks can learn nonlinear representations, which is one reason neural representation learning can outperform PCA when the underlying data manifold is strongly nonlinear.

10.5 Learned latent spaces with neural networks

PCA is an example of a representation-learning method: it replaces the original coordinates with a smaller set of coordinates that preserve a chosen property of the data. Neural networks can do something analogous, but they can learn *nonlinear* transformations and can optimize

the representation for objectives other than variance preservation. This idea is central to representation learning [6, ?, ?].

A useful way to state the idea is to separate an observation from its learned representation. Let

$$\boldsymbol{x} \in \mathbb{R}^d$$

be the original feature vector and let

$$\boldsymbol{z} \in \mathbb{R}^k, \qquad k < d,$$

be a lower-dimensional latent vector.

A neural-network *encoder* is a function

$$\boxed{\boldsymbol{z} = g_\phi(\boldsymbol{x})},$$

where ϕ denotes the encoder's learned parameters. A *decoder* maps the latent representation back toward the original space:

$$\boxed{\widehat{\boldsymbol{x}} = h_\theta(\boldsymbol{z})}$$

with decoder parameters θ.

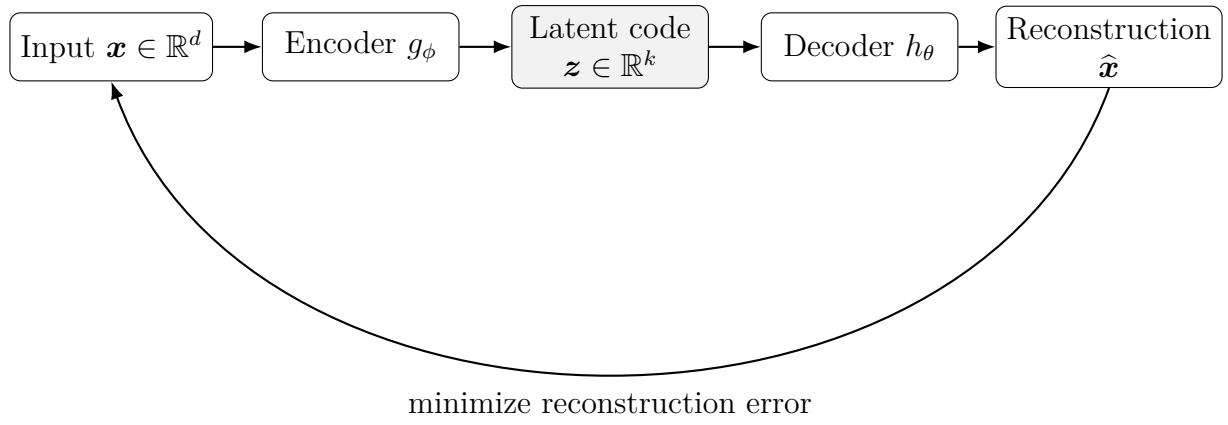


Figure 3: An undercomplete autoencoder. The encoder compresses the input into a k-dimensional latent code, and the decoder learns to reconstruct the original input from that code.

The simplest training objective is mean-squared reconstruction error:

$$\mathcal{L}_{\mathrm{AE}}(\phi, \theta) = \frac{1}{N} \sum_{i=1}^{N} \left\| \boldsymbol{x}^{(i)} - h_\theta\Big(g_\phi(\boldsymbol{x}^{(i)})\Big) \right\|_2^2 .$$

The optimization problem is therefore

$$(\widehat{\phi}, \widehat{\theta}) = \arg\min_{\phi,\theta} \mathcal{L}_{\mathrm{AE}}(\phi, \theta).$$

The notation is important. The network is not merely being asked to "make a small vector." It is being asked to make a small vector from which the original data can still be reconstructed accurately. Information that is useful for reconstruction is therefore encouraged to survive the compression.

> **Expert insight.** A latent space is a learned coordinate system. The coordinates need not correspond one-to-one with the original measured variables. A neural network may instead discover combinations of variables that are useful for reconstructing, predicting, classifying, or generating the data.

10.5.1 Why this is analogous to PCA

PCA finds a linear low-dimensional subspace. For centered data, if

$$W \in \mathbb{R}^{k \times d}$$

has orthonormal rows, the PCA representation is

$$\boldsymbol{z} = W\boldsymbol{x},$$

and the corresponding linear reconstruction is

$$\widehat{\boldsymbol{x}} = W^{\mathsf{T}}\boldsymbol{z} = W^{\mathsf{T}}W\boldsymbol{x}.$$

PCA chooses the subspace so that the average squared reconstruction error is minimized, equivalently so that the retained variance is maximized [10].

An autoencoder can learn a much more general pair of maps,

$$\boldsymbol{x} \xrightarrow{g_\phi} \boldsymbol{z} \xrightarrow{h_\theta} \widehat{\boldsymbol{x}},$$

where g_ϕ and h_θ contain nonlinear activation functions.

Thus PCA and an autoencoder can have the same broad purpose—finding a compact representation—while solving different optimization problems.

There is an especially useful mathematical connection. Under the standard centered-data, squared-error, undercomplete, linear-autoencoder assumptions, a global optimum of the linear autoencoder spans the same principal subspace as PCA, although its individual latent coordinates need not equal the PCA coordinates exactly. The nonlinear autoencoder generalizes this construction by allowing the learned representation to follow curved structure in the data [6, **?**].

10.5.2 What "latent" means

The word *latent* means "not directly observed." The components of $\boldsymbol{z}$ are therefore latent variables constructed by the model rather than quantities that must have been measured directly by an instrument.

This does *not* automatically mean that each latent coordinate corresponds to a unique physical property.

For example, suppose an experiment records

$$\boldsymbol{x} = (x_1, x_2, \ldots, x_8) \in \mathbb{R}^8$$

and an autoencoder maps the data to

$$\boldsymbol{z} = (z_1, z_2) \in \mathbb{R}^2.$$

A useful two-dimensional latent space may make clusters, continuous trends, or other structure visually apparent. However, a rotation such as

$$\boldsymbol{z}' = R\boldsymbol{z}$$

for an appropriate invertible transformation R can describe essentially the same information with different numerical coordinates.

Consequently, latent axes should not automatically be interpreted as named physical quantities without additional evidence or constraints.

> **Important distinction.** Dimensionality reduction is not the same thing as discovering "the true hidden variables." A low-dimensional representation can be useful, predictive, reconstructive, or generative without being uniquely identifiable as a set of physical causes.

10.5.3 Denoising autoencoders: forcing the representation to be useful

A plain autoencoder can sometimes learn an almost trivial identity mapping if its architecture is sufficiently powerful. One important remedy is to deliberately corrupt the input and ask the network to reconstruct the original:

$$\widetilde{\boldsymbol{x}} \sim q(\widetilde{\boldsymbol{x}} \mid \boldsymbol{x}), \qquad \widehat{\boldsymbol{x}} = h_\theta(g_\phi(\widetilde{\boldsymbol{x}}))\,.$$

The loss is then

$$\mathcal{L}_{\mathrm{DAE}} = \frac{1}{N}\sum_{i=1}^{N} \Big\| \boldsymbol{x}^{(i)} - h_\theta\Big(g_\phi(\widetilde{\boldsymbol{x}}^{(i)})\Big) \Big\|_2^2 .$$

Because the network must recover clean structure from an imperfect input, the representation can be encouraged to capture stable features rather than merely copy every input coordinate.

10.5.4 Variational autoencoders: a probabilistic latent space

A variational autoencoder (VAE) changes the interpretation of the latent variables. Instead of producing one deterministic code $\boldsymbol{z}$, the encoder produces a probability distribution for $\boldsymbol{z}$ given $\boldsymbol{x}$:

$$q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) = \mathcal{N}\Big(\boldsymbol{\mu}_\phi(\boldsymbol{x}), \operatorname{diag}\Big(\boldsymbol{\sigma}_\phi^2(\boldsymbol{x})\Big)\Big)\,.$$

A common prior is

$$p(\boldsymbol{z}) = \mathcal{N}(\mathbf{0}, I).$$

The decoder defines a probability model for the data conditioned on the latent variable:

$$p_\theta(\boldsymbol{x} \mid \boldsymbol{z}).$$

The VAE is trained by maximizing the evidence lower bound (ELBO), or equivalently minimizing its negative:

$$\mathcal{L}_{\text{VAE}} = -\mathbb{E}_{q_\phi(\boldsymbol{z}|\boldsymbol{x})} \left[\log p_\theta(\boldsymbol{x} \mid \boldsymbol{z})\right] + D_{\text{KL}} \left(q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z})\right).$$

The first term rewards latent representations that preserve information needed to explain or reconstruct the data. The second term keeps the learned latent distributions close to the chosen prior. This gives the latent space a probabilistic structure and makes sampling from the specified prior part of the model [**?**].

During training, the latent sample can be written as

$$\boxed{\boldsymbol{z} = \boldsymbol{\mu}_\phi(\boldsymbol{x}) + \boldsymbol{\sigma}_\phi(\boldsymbol{x}) \odot \boldsymbol{\epsilon}}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I),$$

where $\odot$ denotes element-by-element multiplication.

This reparameterization makes the stochastic sampling step compatible with gradient-based optimization [**?**].

> **Expert insight.** PCA asks, "What low-dimensional linear coordinates preserve the most variance?"
>
> An autoencoder asks, "What low-dimensional representation lets a neural decoder reconstruct the data well?"
>
> A VAE asks a still different question: "What probabilistically structured latent variables can explain the data well while remaining close to a chosen prior?"
>
> These are related questions, but they are not mathematically identical.

10.5.5 Why latent spaces are useful in science

A learned latent space can be useful when the original parameter space is high-dimensional but the data occupy a much smaller effective set of physically meaningful configurations.

One can then use the latent coordinates for visualization, clustering, anomaly detection, classification, regression, compression, or as an intermediate representation for an inverse problem.

For example, suppose a scientific simulator produces an observation vector $\boldsymbol{x}$ with

$$d = 20$$

measured or derived quantities. A neural encoder might map

$$\boldsymbol{x} \in \mathbb{R}^{20} \longrightarrow \boldsymbol{z} \in \mathbb{R}^{3}.$$

The three latent coordinates can then become inputs to a downstream model,

$$\widehat{y} = r_{\psi}(\boldsymbol{z}),$$

so that the full pipeline is

$$\boldsymbol{x} \xrightarrow{g_{\phi}} \boldsymbol{z} \xrightarrow{r_{\psi}} \widehat{y}.$$

In this sense, representation learning can separate the problem into two conceptual stages:

learn a useful coordinate system $\longrightarrow$ learn the task in that coordinate system.

Important distinction. A learned latent representation should not automatically be interpreted as a unique set of physical causes. Its usefulness for prediction, reconstruction, classification, or inference does not by itself establish physical identifiability.

11 Generalization: why complicated models can still work

A central mystery of modern ML is that highly parameterized models can fit training data extremely closely yet still generalize well in many practical settings. There is no single one-line explanation. Generalization depends on data distribution, architecture, optimization, regularization, noise, parameterization, and inductive bias.

The safest conceptual statement is:

Generalization is an empirical property that must be measured under a specified data-generating scenari

A model is not "general" in the abstract. It generalizes relative to a target population.

11.1 Distribution shift

Let the training distribution be $p_{\text{train}}(x, y)$ and the deployment distribution be $p_{\text{deploy}}(x, y)$. If

$$p_{\text{train}} \neq p_{\text{deploy}},$$

performance can change even when the model and algorithm are unchanged.

This is particularly important in science: a simulation may be accurate over one region of physical parameter space while observations occupy another.

12 A compact survey of common ML algorithms

Neural networks are not universally superior. Different algorithms have different inductive biases, data requirements, computational costs, and strengths.

Method	Basic idea	Typical strengths / contexts	Typical limitation
Linear / logistic regression	Fit a parametric linear function or log-odds model	Fast, interpretable, excellent baseline	Limited nonlinear structure
Decision tree	Recursively split feature space into regions	Interpretable rules; handles nonlinear thresholds	Can overfit; unstable alone
Random forest	Average many randomized decision trees	Strong tabular baseline; robust and flexible	Usually not ideal for raw high-dimensional unstructured data
Gradient-boosted trees	Build trees sequentially to correct previous errors	Excellent for many tabular-data problems	More tuning; sequential training
k-nearest neighbors	Predict from nearby training points	Simple, local, useful for small data and diagnostics	Distance becomes less informative in high dimension
Support vector machine	Find a margin-maximizing decision boundary, possibly with kernels	Effective in moderate-dimensional classification/regression	Scaling can be difficult for huge datasets
k-means	Represent data by nearest cluster center	Simple unsupervised clustering	Assumes useful geometry and a chosen number of clusters
Spectral clustering	Use eigenstructure of a similarity graph to reveal groups	Useful for non-convex cluster geometry and graph-like relationships	Can be computationally expensive; similarity choice matters
PCA	Rotate coordinates to directions of maximal variance	Dimensionality reduction, visualization, denoising	Only linear; high variance need not equal useful predictive information
Neural network	Learn a composition of nonlinear transformations	Images, audio, language, complex scientific mappings, large datasets	Data/compute hungry; less interpretable; tuning and validation are essential
Gaussian process	Define a probability distribution over functions	Small-to-medium data with principled uncertainty	Computational cost grows rapidly with data size

Random forests were introduced by Breiman as ensembles of randomized tree predictors whose generalization behavior improves through averaging and decorrelation of trees [11]. Spectral clustering uses the eigenstructure of a similarity matrix/graph to produce useful cluster representations; it is valuable when "nearby in feature space" is better expressed by a graph than by simple spherical clusters.

> **Important distinction.** There is no universally best ML algorithm. "Better" must be tied to a problem, data regime, compute budget, objective, uncertainty requirement, interpretability requirement, and evaluation protocol.

13 Simulation-based inference and simulation-based calibration

Many scientific inverse problems have the following structure:

$$\theta \longrightarrow \text{physical simulator} \longrightarrow x,$$

where θ represents physical parameters and x represents observable data.

The forward simulator is often available, but the likelihood

$$p(x \mid \theta)$$

may be expensive or impossible to evaluate analytically.

Simulation-based inference (SBI) uses simulations to learn quantities needed for inference, such as an approximate posterior,

$$p(\theta \mid x),$$

an approximate likelihood, or a likelihood ratio.

13.1 Inverse problem intuition

Suppose a simulator maps

$$\theta \mapsto x = g(\theta, \epsilon),$$

where ϵ represents stochasticity.

Given observed data x_{obs}, the scientific problem is reversed:

$$x_{\text{obs}} \longrightarrow \text{what values of } \theta \text{ are plausible?}$$

This is often harder than the forward problem.

13.2 Normalizing flows

A normalizing flow constructs a flexible probability density by transforming a simple reference random variable $\boldsymbol{z}$ into a more complicated variable $\boldsymbol{x}$ through an invertible map

$$\boldsymbol{x} = f_\phi(\boldsymbol{z}).$$

The change-of-variables formula is

$$p_X(\boldsymbol{x}) = p_Z(\boldsymbol{z}) \left| \det \frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} \right|.$$

Equivalently, using $\boldsymbol{z} = f_\phi^{-1}(\boldsymbol{x})$,

$$\log p_X(\boldsymbol{x}) = \log p_Z(\boldsymbol{z}) + \log \left| \det \frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} \right|.$$

A neural network can be used to parameterize the invertible transformation. This connects deep learning to rigorous probability theory rather than replacing probability with a purely black-box predictor.

13.3 Simulation-based calibration (SBC)

When a Bayesian inference method is intended to recover posterior distributions, calibration can be tested by generating synthetic parameters from the prior and synthetic data from the simulator, then checking whether the inference procedure statistically recovers the known generating parameters.

A simplified SBC experiment is:

1. draw $\theta^{(s)} \sim p(\theta)$;
2. simulate $x^{(s)} \sim p(x \mid \theta^{(s)})$;
3. run the inference procedure on $x^{(s)}$ to obtain an approximate posterior;
4. check where the known $\theta^{(s)}$ falls within the inferred posterior samples;
5. repeat many times and test whether the resulting rank statistics behave as expected under calibration.

The key idea is extraordinarily powerful: *because the truth is known in simulation, the inference machinery can be tested end-to-end.*

14 Examples from astrophysics

The following examples illustrate how the general ML ideas above enter real scientific work. They are presented as applications of the concepts, not as claims that any one ML method is universally optimal.

14.1 1. Simulation-based calibrated Bayesian inference for binary stellar evolution

A binary stellar-evolution simulation can map physical assumptions and initial conditions into predicted observable outcomes. If several stages of the inference pipeline are approximate, then an apparently plausible posterior can still be miscalibrated.

A natural workflow is:

$$\text{physical parameters} \to \text{binary-evolution simulator} \to \text{synthetic observations} \to \text{inference} \to \text{calibration test}.$$

The scientific lesson is broader than the particular binary-evolution application: reproducibility is not just "can another person run the code?" It also includes whether the inferred scientific conclusions remain stable when the stated model, simulator, and calibration procedure are reconstructed independently.

14.2 2. Normalizing flows and simulation-based inference for multi-messenger inverse problems

Multimessenger astronomy combines heterogeneous observations, for example gravitational-wave and electromagnetic information, to constrain common physical parameters.

The inverse problem can be written schematically as

$$\theta \to x_{\mathrm{GW}}, x_{\mathrm{EM}},$$

followed by

$$(x_{\mathrm{GW}}, x_{\mathrm{EM}}) \to p(\theta \mid x_{\mathrm{GW}}, x_{\mathrm{EM}}).$$

Normalizing flows are useful because they can represent flexible non-Gaussian probability densities while retaining an explicit probabilistic formulation. The combination with simulation-based inference lets a computational model learn how simulated observations relate to latent physical parameters.

14.3 3. ML classification in the black-hole mass–spin plane

Consider a feature vector

$$\boldsymbol{x} = \begin{bmatrix} M_{\mathrm{BH}} \\ \chi \end{bmatrix},$$

where M_{BH} is black-hole mass and χ is a spin parameter.

The task may be to identify populations in this two-dimensional parameter space. One can use unsupervised clustering to search for structure and supervised classifiers when labeled examples or simulated population assignments are available.

A recent published example applies unsupervised clustering, representation learning with variational autoencoders, and supervised random forests to mass–spin data, demonstrating that the choice of representation and method can matter when populations overlap [12].

> **Expert insight.** This is the same story as the neural-network example: the algorithm is only one layer of the scientific argument. A reliable result requires

a meaningful representation, a well-defined target, appropriate training data, validation, and an assessment of how the result changes under plausible shifts in the data-generating assumptions.

15 From machine learning to today's large language models

15.1 Tokens

An LLM does not read text as a human does. Text is converted into discrete tokens. A token might be a whole word, a word fragment, punctuation, or another text unit.

If a token sequence is

$$t_1, t_2, \ldots, t_T,$$

the model can be trained to estimate

$$p(t_k \mid t_1, \ldots, t_{k-1}).$$

The sequence probability factorizes as

$$p(t_1, \ldots, t_T) = \prod_{k=1}^{T} p(t_k \mid t_1, \ldots, t_{k-1}).$$

Taking logarithms gives

$$\log p(t_1, \ldots, t_T) = \sum_{k=1}^{T} \log p(t_k \mid t_{<k}),$$

which is why next-token cross-entropy is a natural training objective.

15.2 Embeddings

Each token is mapped to a vector in a high-dimensional space:

$$t_k \mapsto \boldsymbol{e}_k \in \mathbb{R}^d.$$

The geometry of this representation is learned rather than hand-written.

15.3 Attention

The Transformer introduced scaled dot-product attention. Given query, key, and value matrices Q, K, V,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}}\right) V.$$

The matrix QK^{T} measures pairwise compatibility. Division by $\sqrt{d_k}$ stabilizes the scale of the dot products as the dimension grows. Softmax converts each row into weights that sum to one.

A simple one-row interpretation is:

$$\alpha_j = \frac{e^{s_j}}{\sum_k e^{s_k}},$$

so the output is a weighted combination

$$\sum_j \alpha_j \boldsymbol{v}_j.$$

15.4 Why the Transformer matters

Attention permits each token representation to incorporate information from other positions in the sequence. The original Transformer paper demonstrated strong sequence-modeling performance while being more parallelizable than recurrent architectures [7].

15.5 Scale

Large language models are empirically sensitive to model size, data size, and compute. Scaling-law work found systematic relationships between cross-entropy loss and these resources over broad regimes [9]. This does not imply that simply making every model bigger always solves every problem; data quality, architecture, optimization, evaluation, and inference method all matter.

15.6 Human feedback and instruction following

A pretrained model is optimized to predict text, not necessarily to follow user instructions in a desirable way. A post-training procedure can use human-written demonstrations and human preferences over candidate answers. InstructGPT is a well-known demonstration of supervised fine-tuning plus RLHF for improving instruction-following behavior [8].

16 What other kinds of AI exist?

AI predates modern deep learning. Important families of approaches include:

- symbolic logic and rule systems;
- search and planning algorithms;
- knowledge representation systems;
- probabilistic graphical models and Bayesian inference;
- evolutionary and population-based optimization;
- control-theoretic and decision-theoretic systems;
- reinforcement-learning systems;
- neural networks and deep learning.

These approaches can also be combined. For example, a modern AI product may contain a neural language model plus symbolic constraints, retrieval, deterministic software, search, and a conventional database.

The phrase "Bayesian brain" refers to a family of theoretical ideas in which perception or cognition is modeled as probabilistic inference. Bayesian reasoning is important in AI and machine learning, but a Bayesian formulation by itself does not specify a competitive chatbot architecture.

> **Important distinction.** Modern frontier chatbots are ML-based, but it is inaccurate to conclude that every AI system—either historically or today—must contain a neural network or machine-learning component.

17 AI competition, current systems, and the US–China comparison

AI development is also a technological and economic competition among companies and countries. The relevant evidence changes rapidly, so this section is intentionally time-stamped.

As reported in Stanford HAI's 2026 AI Index, U.S. and Chinese frontier-model performance has alternated at the top of several evaluations since early 2025. The report states that by March 2026 the leading U.S. model led the leading Chinese model by about 2.7% on its chosen aggregate measure, while the gap had remained small and fluctuated over the preceding year [13]. The same report describes the United States as continuing to lead in private AI investment, while China has been very strong in publications, citations, patents, and industrial deployment measures; the details depend on the metric and year considered [14].

These facts support a careful conclusion: there is a genuine and ongoing U.S.–China competition in AI capability, investment, infrastructure, and deployment. They do *not* by themselves imply that one country "owns" AI or that technical leadership is permanent.

18 Should we fear AI? A scientifically useful way to discuss the question

Fear is not a scientific variable, so a classroom treatment should separate three questions:

1. What can current systems actually do?
2. What risks can follow from those capabilities and their deployment?
3. Which risks are created by models themselves, and which arise from human incentives, institutions, misuse, or poor engineering?

NIST emphasizes that AI systems can introduce risks affecting individuals, organizations, communities, society, and the environment, and it recommends structured risk management rather than relying on optimism or fear alone [3, 4].

A reasonable discussion position is therefore:

> *We should neither assume that AI is harmless nor assume that catastrophic outcomes are inevitable. We should measure capabilities, identify plausible failure modes, build safeguards, and pay attention to the people and institutions deploying the technology.*

One provocative classroom formulation is that humanity often adapts to powerful technologies by normalizing them and embedding them into institutions. Nuclear weapons are an example of a technology whose existence did not cause society to stop functioning, but they also illustrate why normalization should not be confused with safety. AI deserves the same combination of technical study, institutional caution, and historical perspective.

The important educational point is that "AI risk" is not a single variable. Different risks have different mechanisms:

$$\begin{aligned}&\text{misinformation}, \quad \text{privacy loss}, \quad \text{bias},\\ &\text{cybersecurity misuse}, \quad \text{economic disruption},\\ &\text{loss of human oversight}, \quad \text{model unreliability}, \ldots\end{aligned}$$

Each must be evaluated with the appropriate evidence.

19 A unifying picture: ML as function learning

Nearly everything in these notes can be connected to one abstract pattern.

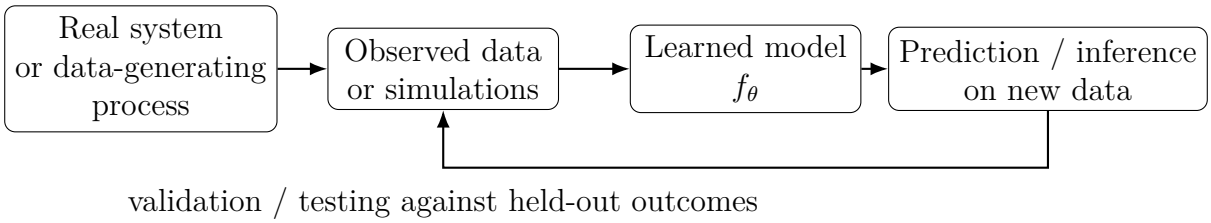


Figure 4: The unifying ML workflow. The learned model is not the world itself; it is an inferred representation designed for a specified task. Validation and testing evaluate the learned mapping on data not used for fitting its parameters.

The strongest conceptual connection to physics is this: a physical model is useful because it captures a stable structure of the world. A machine-learning model is useful when its learned representation captures structure that remains informative outside the exact examples used in training.

> **Expert insight.** The phrase "the network learned" should always be followed by a question: *what, exactly, did it learn, under what data-generating assumptions, and how do we know?* That question is the bridge between computation and science.

20 Compact summary for students

Idea	One-sentence definition
AI	The broad field of building artificial systems that perform tasks associated with intelligence.
ML	Methods that infer useful mappings, structures, or distributions from data or experience.
Supervised learning	Learn from examples with target outputs.
Unsupervised learning	Discover structure without externally supplied target labels.
Self-supervised learning	Create prediction targets automatically from the input data.
Regression	Predict a numerical quantity.
Classification	Predict a discrete category.
Neural network	A composition of parameterized functions joined by learned weights and nonlinear activations.
Training	Adjust model parameters to reduce a loss on training data.
Generalization	Performing well on new data from the target population.
Overfitting	Fitting idiosyncrasies of the training data that do not transfer reliably.
PCA	Linear dimensionality reduction based on directions of maximal variance.
SBI	Use simulations to learn or approximate quantities needed for inference when direct likelihood evaluation is difficult.
LLM	A large neural language model trained primarily to model token sequences, usually by next-token prediction and subsequent post-training.

21 Key equations

$$\begin{aligned}
\text{Linear neuron:} &\quad z = \boldsymbol{w} \cdot \boldsymbol{x} + b. \\
\text{Sigmoid:} &\quad \sigma(z) = \frac{1}{1 + e^{-z}}. \\
\text{Regression MSE:} &\quad J = \frac{1}{N} \sum_i (\widehat{y}_i - y_i)^2. \\
\text{Binary cross-entropy:} &\quad J = -\frac{1}{N} \sum_i \left[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \right]. \\
\text{Gradient descent:} &\quad \boldsymbol{\theta}_{\text{new}} = \boldsymbol{\theta}_{\text{old}} - \eta \nabla J. \\
\text{Covariance matrix:} &\quad C = \frac{1}{N - 1} \sum_i \boldsymbol{x}_i \boldsymbol{x}_i^{\mathsf{T}}. \\
\text{PCA eigenproblem:} &\quad C\boldsymbol{u} = \lambda \boldsymbol{u}. \\
\text{Bayes' theorem:} &\quad p(\theta \mid D) = \frac{p(D \mid \theta) p(\theta)}{p(D)}. \\
\text{Change of variables:} &\quad p_X(x) = p_Z(z) \left| \det \frac{\partial z}{\partial x} \right|. \\
\text{Transformer attention:} &\quad \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}} \right) V.
\end{aligned}$$

22 Questions for discussion or assessment

1. Why is "AI = machine learning" too strong a statement historically and technically?
2. Why is self-supervised learning a useful description of next-token prediction?
3. Give an example in which a model has low training error but poor generalization.
4. Explain why a neural network with only linear layers is still just a linear/affine mapping.
5. Why can a well-calibrated classifier be more informative than a classifier judged only by accuracy?
6. Why is high-dimensional parameter-space coverage difficult?
7. What information is lost when a physical system is represented by too few features?
8. Why can PCA reduce dimension without necessarily improving the final prediction task?
9. Explain the difference between a forward simulator and an inverse inference problem.
10. Why is simulation-based calibration possible when the real-world truth is usually unknown?

11. What does attention compute mathematically, and why does it involve a softmax?
12. Why does "a bigger neural network" not automatically mean "a better scientific model"?

References

References

[1] Stanford Encyclopedia of Philosophy, *Artificial Intelligence*, first published 2018, substantially revised later. https://plato.stanford.edu/entries/artificial-intelligence/

[2] Stanford Encyclopedia of Philosophy, *Logic-Based Artificial Intelligence*, substantially revised 2024. https://plato.stanford.edu/entries/logic-ai/

[3] E. Tabassi, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, National Institute of Standards and Technology, 2023. https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-ai-rmf-10

[4] National Institute of Standards and Technology, *AI RMF 1.0 Executive Summary*, 2023. https://airc.nist.gov/airmf-resources/airmf/0-ai-rmf-1-0/

[5] T. M. Mitchell, *Machine Learning*, McGraw–Hill, 1997.

[6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016. https://www.deeplearningbook.org/

[7] A. Vaswani et al., "Attention Is All You Need," 2017. https://arxiv.org/abs/1706.03762

[8] L. Ouyang et al., "Training language models to follow instructions with human feedback," 2022. https://arxiv.org/abs/2203.02155

[9] J. Kaplan et al., "Scaling Laws for Neural Language Models," 2020. https://arxiv.org/abs/2001.08361

[10] I. T. Jolliffe, *Principal Component Analysis*, 2nd ed., Springer, 2002. DOI: https://doi.org/10.1007/b98835

[11] L. Breiman, "Random Forests," *Machine Learning* 45, 5–32 (2001). DOI: https://doi.org/10.1023/A:1010933404324

[12] N. Steinle and S. Safi-Harb, "Machine learning classification of black holes in the mass–spin diagram," 2025. https://arxiv.org/abs/2508.14316

[13] Stanford Institute for Human-Centered Artificial Intelligence, *2026 AI Index Report: Technical Performance*, 2026. https://hai.stanford.edu/ai-index/2026-ai-index-report/technical-performance

[14] Stanford Institute for Human-Centered Artificial Intelligence, *2026 AI Index Report: Economy*, 2026. https://hai.stanford.edu/ai-index/2026-ai-index-report/economy

Note on the astrophysics examples

The three astrophysics examples in these notes were included to connect the general ML framework to the instructor's research applications: (1) simulation-based calibrated Bayesian inference for binary stellar-evolution simulations, (2) normalizing flows and simulation-based inference for multimessenger inverse problems, and (3) machine-learning classification of black-hole populations in the mass–spin diagram. The third example is additionally linked to the published paper cited above. Full bibliographic details for the first two instructor-specific examples were not supplied in the prompt, so no invented paper titles or author lists have been added here.